

LAPORAN TEKNIS PROYEK AKHIR
MATA KULIAH ALGORITMA DAN PEMROGRAMAN

TokoNet: Sistem Manajemen Toko Berbasis Client-Server



Dosen Pengampu:
Alfan Presekai, ST., M.Sc., Ph.D.

Disusun Oleh:

Altafico Fattan Wildana	2506591186
Josias Shihkai Nazaro Sitanggang	2506609435
Fahrul Gustian Rezky	2506585510

UNIVERSITAS INDONESIA
2026

DAFTAR ISI

DAFTAR ISI	2
BAB 1. PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan Proyek.....	4
1.3 Ruang Lingkup	4
BAB 2. DESKRIPSI PROYEK	5
2.1 Gambaran Umum Sistem	5
2.2 Komponen Sistem	5
2.3 Fitur Sistem.....	6
BAB 3. DESAIN SISTEM DAN JARINGAN	6
3.1 Arsitektur Client-Server.....	6
3.2 Protokol Komunikasi	6
3.3 Format Pertukaran Data JSON	7
3.4 Alur Komunikasi.....	8
BAB 4. STRUKTUR DATA: LINKED LIST MANUAL	8
4.1 Alasan Pemilihan Linked List	8
4.2 Struktur Node	8
4.3 Kelas LinkedList	9
4.4 Analisis Kompleksitas Operasi Linked List	10
BAB 5. IMPLEMENTASI ALGORITMA	10
5.1 Linear Search	10
5.1.1 Alasan Pemilihan	10
5.1.2 Cara Kerja	10
5.1.3 Analisis Kompleksitas	11
5.2 Quick Sort	11
5.2.1 Alasan Pemilihan	11
5.2.2 Cara Kerja	12
5.2.3 Analisis Kompleksitas	12
BAB 6. PARADIGMA PEMROGRAMAN BERORIENTASI OBJEK	13
6.1 Abstraksi	13
6.2 Enkapsulasi	13
6.3 Pewarisan	14

6.4 Polimorfisme	15
BAB 7. MULTITHREADING DAN SINKRONISASI	15
7.1 Kebutuhan Multithreading	15
7.2 Implementasi Thread per Client.....	16
7.3 Race Condition dan Kebutuhan Sinkronisasi.....	16
7.4 Mutex sebagai Mekanisme Sinkronisasi	17
BAB 8 DIAGRAM ALUR SISTEM	17
8.1 Alur Kerja Server.....	18
8.2 Alur Kerja Client.....	20
8.3 Alur Transaksi Penjualan	22
BAB 9 KESIMPULAN	23

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong berbagai sektor usaha untuk mengadopsi sistem pengelolaan berbasis komputer, termasuk sektor perdagangan ritel. Sistem kasir konvensional yang bersifat terpusat pada satu perangkat memiliki keterbatasan dalam hal skalabilitas dan efisiensi operasional, terutama ketika terdapat lebih dari satu titik layanan yang harus dilayani secara bersamaan.

Proyek TokoNet hadir sebagai solusi sistem manajemen toko berbasis arsitektur client-server yang memungkinkan beberapa kasir mengakses dan memanipulasi data inventori secara bersamaan melalui jaringan TCP/IP. Sistem ini dirancang dengan menerapkan prinsip-prinsip pemrograman berorientasi objek (OOP), struktur data dinamis manual, serta algoritma pencarian dan pengurutan yang diimplementasikan secara mandiri tanpa bergantung pada pustaka bawaan.

1.2 Tujuan Proyek

Proyek ini bertujuan untuk mengintegrasikan dan mendemonstrasikan penguasaan konsep-konsep fundamental dalam mata kuliah Algoritma dan Pemrograman, meliputi:

1. Mengimplementasikan paradigma pemrograman berorientasi objek secara menyeluruh melalui keempat pilarnya, yaitu abstraksi, enkapsulasi, pewarisan, dan polimorfisme, dalam konteks sistem yang nyata dan fungsional.
2. Membangun struktur data Linked List secara manual tanpa menggunakan pustaka standar C++ seperti `std::list` atau `std::vector`, sehingga membuktikan pemahaman mendalam tentang manajemen memori dinamis dan operasi pointer.
3. Mengimplementasikan algoritma pencarian (Linear Search) dan pengurutan (Quick Sort) secara mandiri lengkap dengan analisis kompleksitasnya dalam notasi Big O.
4. Membangun sistem komunikasi jaringan berbasis TCP Socket antara server dan client dengan format pertukaran data JSON yang disusun secara manual.
5. Mengimplementasikan server yang mampu menangani lebih dari satu client secara bersamaan menggunakan mekanisme multithreading yang dilindungi sinkronisasi mutex.

1.3 Ruang Lingkup

Sistem TokoNet mencakup pengelolaan inventori toko yang terdiri dari tiga kategori barang (Makanan, Elektronik, dan Minuman), dengan fitur utama berupa tampilan katalog, pengurutan berdasarkan harga, dan transaksi penjualan. Sistem dibangun menggunakan bahasa C++ standar (C++11) dengan pustaka POSIX untuk socket programming, dan dijalankan pada lingkungan Linux/WSL.

BAB 2. DESKRIPSI PROYEK

2.1 Gambaran Umum Sistem

TokoNet adalah sistem manajemen toko berbasis jaringan yang terdiri dari dua program terpisah: server dan client. Server berjalan sebagai proses latar belakang yang menyimpan seluruh data inventori dan mengeksekusi logika bisnis, sementara client berfungsi sebagai antarmuka kasir yang menerima input pengguna, mengirimkan permintaan ke server, dan menampilkan hasil kepada pengguna.

Interaksi antara kedua komponen sepenuhnya menggunakan protokol TCP (Transmission Control Protocol) yang menjamin pengiriman data secara berurutan dan terpercaya. Seluruh data yang dikirimkan antara client dan server dikemas dalam format JSON (JavaScript Object Notation) yang disusun secara manual menggunakan operasi string dasar C++.

2.2 Komponen Sistem

Sistem TokoNet terdiri dari tiga file utama yang saling berkaitan. Berikut adalah rincian masing-masing komponen:

File	Peran	Deskripsi
shared_inventory.h	Header Bersama	Inti dari seluruh sistem. Berisi definisi namespace Json untuk utilitas pemrosesan JSON, hierarki kelas OOP (abstract class Item beserta subclass-nya), serta implementasi Linked List manual lengkap dengan algoritma pencarian dan pengurutan. Di-include oleh kedua program server dan client.
server.cpp	Program Server	Mengelola data inventori secara terpusat. Menerima koneksi dari client, memproses setiap permintaan dalam thread terpisah, melakukan pencarian dan modifikasi data inventori, serta mengirimkan respons dalam format JSON. Mendukung banyak client secara bersamaan melalui mekanisme multithreading.
client.cpp	Program Client	Antarmuka kasir yang menyediakan menu interaktif berbasis terminal. Mengirimkan permintaan ke server dalam format JSON dan menampilkan respons yang diterima dalam bentuk tabel yang terstruktur rapi.

2.3 Fitur Sistem

Sistem TokoNet menyediakan empat fitur utama yang dapat diakses melalui menu interaktif client, sebagaimana dirangkum dalam tabel berikut:

No.	Fitur	Deskripsi
1	Lihat Semua Barang	Menampilkan seluruh daftar inventori dalam format tabel beserta informasi nama, tipe, harga, stok, dan persentase diskon masing-masing barang.
2	Lihat Barang Urut Harga	Menampilkan daftar inventori setelah diurutkan berdasarkan harga dari yang terendah hingga tertinggi menggunakan algoritma Quick Sort.
3	Transaksi Penjualan	Memungkinkan kasir melakukan penjualan dengan memilih nama barang dan jumlah yang dibeli. Sistem secara otomatis mengurangi stok, menghitung total harga setelah diskon, dan melaporkan sisa stok.
4	Validasi & Penanganan Kesalahan	Memberikan respons informatif ketika barang tidak ditemukan atau stok tidak mencukupi permintaan.

BAB 3. DESAIN SISTEM DAN JARINGAN

3.1 Arsitektur Client-Server

Sistem TokoNet mengadopsi arsitektur client-server dua tingkat (two-tier architecture) di mana logika bisnis dan penyimpanan data sepenuhnya dikelola oleh server, sementara client hanya bertanggung jawab atas presentasi dan pengambilan input pengguna. Pemisahan tanggung jawab ini menghasilkan sistem yang mudah diskalakan karena penambahan kasir baru hanya memerlukan penjalankan program client tambahan tanpa perlu mengubah konfigurasi server.

Server berjalan secara persisten dan mendengarkan koneksi masuk pada port 8080. Setiap client yang terhubung akan dilayani secara independen oleh thread tersendiri, sehingga aktivitas satu kasir tidak memblokir kasir lainnya. Data inventori disimpan secara terpusat di dalam Linked List yang dikelola oleh server dan dapat diakses oleh semua thread melalui mekanisme sinkronisasi mutex.

3.2 Protokol Komunikasi

Komunikasi antara client dan server menggunakan protokol TCP (Transmission Control Protocol) melalui POSIX Socket API. TCP dipilih atas UDP karena beberapa pertimbangan teknis yang krusial untuk aplikasi manajemen toko. TCP menjamin bahwa setiap paket data sampai ke tujuan secara utuh dan berurutan. Hal ini sangat penting dalam konteks transaksi penjualan di mana kehilangan atau kerusakan satu paket permintaan dapat mengakibatkan inkonsistensi data stok.

Selain itu, TCP menyediakan mekanisme kontrol aliran dan deteksi kesalahan yang memastikan integritas data tanpa perlu implementasi tambahan di level aplikasi.

Untuk menginisialisasi koneksi, server mengikuti urutan pemanggilan fungsi yang bersifat wajib. Urutan tersebut dimulai dari `socket()` untuk membuat descriptor socket baru, dilanjutkan `setsockopt()` dengan opsi `SO_REUSEADDR` agar port dapat segera digunakan kembali setelah server direstart, kemudian `bind()` untuk mendaftarkan socket ke port 8080, lalu `listen()` untuk mulai menerima antrian koneksi, dan terakhir `accept()` dalam sebuah loop untuk menerima setiap client baru yang masuk. Di sisi client, urutan yang digunakan adalah `socket()`, `connect()`, `send()`, `recv()`, dan `close()`.

Berbeda dari arsitektur persistent connection, client pada sistem ini menggunakan pola stateless connection di mana setiap permintaan membuka koneksi baru, menyelesaikan transaksi, lalu menutup koneksi tersebut. Pola ini dipilih karena kesederhanaannya dan kemampuannya menghindari masalah koneksi yang tersangkut akibat gangguan jaringan.

3.3 Format Pertukaran Data JSON

Seluruh komunikasi antara client dan server menggunakan format JSON (JavaScript Object Notation) yang disusun dan diparsing secara manual menggunakan operasi string standar C++. Pemilihan JSON didasarkan pada keterbacaannya oleh manusia sehingga memudahkan proses debugging, serta struktur key-value yang sederhana dan mudah diparsing menggunakan fungsi `find()` dan `substr()` tanpa memerlukan pustaka eksternal.

Terdapat tiga jenis permintaan (request) yang dapat dikirimkan client kepada server:

Jenis Permintaan	Format JSON
Lihat inventori (tanpa urutan)	<code>{"aksi":"lihat","namaBarang":"","jumlah":0}</code>
Lihat inventori (urut harga)	<code>{"aksi":"lihat_urut","namaBarang":"","jumlah":0}</code>
Transaksi penjualan	<code>{"aksi":"jual","namaBarang":"Aqua 600ml","jumlah":3}</code>

Terdapat tiga jenis respons yang dikirimkan server kepada client:

Jenis Respons	Format JSON
Transaksi berhasil	<code>{"status":"sukses","sisaStok":97,"totalHarga":11640,"diskon":3}</code>

Jenis Respons	Format JSON
Kegagalan	<code>{"status":"gagal","pesan":"Stok tidak cukup. Tersedia: 2"}</code>
Daftar inventori	<code>{"status":"sukses","daftar":[{"nama":"Aqua 600ml","tipe":"Minuman","harga":4000,"stok":97,"diskon":3},...]}</code>

Parser JSON yang diimplementasikan secara manual menggunakan dua fungsi utama. Fungsi `getString()` menemukan posisi key menggunakan `find()`, mencari tanda titik dua setelahnya, lalu mengekstrak nilai teks di antara dua tanda kutip menggunakan `substr()`. Fungsi `getAngka()` bekerja serupa namun mengekstrak nilai numerik tanpa tanda kutip dengan mendeteksi karakter digit secara berurutan.

3.4 Alur Komunikasi

Setiap interaksi dalam sistem mengikuti pola yang konsisten. Kasir memilih opsi menu pada program client, lalu client menyusun permintaan dalam format JSON dan mengirimkannya ke server melalui koneksi TCP yang baru dibuka. Server menerima data melalui fungsi `recv()` yang bersifat blocking, memarsing field "aksi" dari JSON untuk menentukan jenis permintaan, meneruskan ke fungsi pemroses yang sesuai (`prosesLihat`, `prosesLihatUrut`, atau `prosesJual`), dan mengirimkan respons JSON melalui fungsi `send()`. Di sisi client, respons diterima, diparsing, dan ditampilkan dalam format yang terstruktur kepada kasir.

BAB 4. STRUKTUR DATA: LINKED LIST MANUAL

4.1 Alasan Pemilihan Linked List

Linked List dipilih sebagai struktur data utama untuk menyimpan inventori dengan beberapa pertimbangan. Berbeda dengan array yang memiliki ukuran tetap, Linked List bersifat dinamis sehingga memungkinkan penambahan dan penghapusan elemen kapan saja tanpa perlu mengalokasikan ulang seluruh memori. Hal ini relevan untuk sistem toko yang jumlah jenis barangnya dapat berubah sewaktu-waktu. Selain itu, operasi penambahan dan penghapusan pada Linked List tidak memerlukan pergeseran elemen seperti pada array, sehingga lebih efisien untuk koleksi yang sering dimodifikasi.

Implementasi Linked List pada sistem ini dilakukan secara manual tanpa menggunakan pustaka bawaan `std::list` maupun `std::vector`. Keputusan ini diambil untuk mendemonstrasikan pemahaman mendalam tentang manajemen memori dinamis menggunakan operator `new` dan `delete`, serta manipulasi pointer secara langsung.

4.2 Struktur Node

Unit dasar dari Linked List adalah `struct Node` yang terdiri dari dua anggota:

Anggota	Tipe	Keterangan
data	Item*	Pointer yang menyimpan alamat memori objek barang. Menggunakan pointer ke kelas induk Item (bukan objek langsung) agar dapat menerapkan polimorfisme — satu Node dapat menyimpan objek bertipe Makanan, Elektronik, maupun Minuman.
next	Node*	Pointer yang menyimpan alamat Node berikutnya dalam rantai. Bernilai nullptr pada Node terakhir sebagai penanda akhir list.

4.3 Kelas LinkedList

Kelas LinkedList menyimpan pointer head yang menunjuk ke Node pertama dan variabel ukuran yang mencatat jumlah elemen saat ini. Kelas ini menyediakan beberapa operasi fundamental:

Operasi	Kompleksitas	Deskripsi
tambah()	$O(n)$	Menambahkan item baru di akhir list. Membuat Node baru di heap menggunakan operator new, kemudian menelusuri list dari head hingga Node terakhir dan menghubungkan Node baru tersebut.
cari()	$O(n)$	Mengimplementasikan Linear Search untuk menemukan barang berdasarkan nama. Dibahas mendalam pada Bab 5.
urutkanHarga()	$O(n \log n)$	Mengimplementasikan Quick Sort untuk mengurutkan inventori berdasarkan harga. Dibahas mendalam pada Bab 5.
toJsonArray()	$O(n)$	Menelusuri seluruh list dan memanggil metode toJson() pada setiap item untuk menghasilkan representasi JSON seluruh inventori, memanfaatkan polimorfisme.
~LinkedList()	$O(n)$	Destruktor yang membebaskan seluruh memori yang dialokasikan secara dinamis, mencegah memory leak dengan menghapus setiap Node dan objek Item-nya.

4.4 Analisis Kompleksitas Operasi Linked List

Berikut adalah analisis kompleksitas waktu dan ruang untuk setiap operasi pada implementasi Linked List dalam sistem ini:

Operasi	Best Case	Average Case	Worst Case	Space
tambah()	$O(n)$	$O(n)$	$O(n)$	$O(1)$
cari()	$O(1)$	$O(n)$	$O(n)$	$O(1)$
urutkanHarga()	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
tampilSemua()	$O(n)$	$O(n)$	$O(n)$	$O(1)$
toJsonArray()	$O(n)$	$O(n)$	$O(n)$	$O(1)$

BAB 5. IMPLEMENTASI ALGORITMA

5.1 Linear Search

5.1.1 Alasan Pemilihan

Linear Search dipilih sebagai algoritma pencarian untuk menemukan barang berdasarkan nama dalam Linked List. Meski Binary Search menawarkan kompleksitas $O(\log n)$ yang lebih baik, Binary Search mensyaratkan adanya akses acak (random access) ke elemen di posisi tengah, yang lazim dilakukan pada array dengan notasi $arr[n/2]$. Linked List tidak mendukung akses acak ini karena untuk mencapai node ke- k , program harus menelusuri dari head satu per satu melalui pointer next. Mengonversi Linked List ke array terlebih dahulu hanya untuk menerapkan Binary Search justru menambah overhead $O(n)$ waktu dan ruang tanpa keuntungan yang berarti. Dengan demikian, Linear Search merupakan pilihan yang paling tepat dan efisien secara praktis untuk struktur data Linked List.

5.1.2 Cara Kerja

Fungsi `cari()` memulai penelusuran dari Node pertama (head). Pada setiap Node, nilai nama barang yang disimpan dibandingkan dengan nama yang dicari menggunakan operator perbandingan string. Jika ditemukan kecocokan, fungsi langsung mengembalikan pointer ke objek Item yang bersangkutan. Jika penelusuran telah mencapai akhir list tanpa menemukan kecocokan, fungsi mengembalikan nilai `nullptr` sebagai penanda bahwa barang tidak ditemukan.

5.1.3 Analisis Kompleksitas

Kasus	Kompleksitas Waktu	Kompleksitas Ruang	Kondisi
Best Case	$O(1)$	$O(1)$	Barang yang dicari berada pada Node pertama sehingga penelusuran selesai setelah satu kali perbandingan.
Average Case	$O(n)$	$O(1)$	Secara statistik, jika elemen yang dicari ada dalam list, rata-rata posisinya berada di tengah sehingga diperlukan sekitar $n/2$ perbandingan.
Worst Case	$O(n)$	$O(1)$	Barang tidak ada dalam list, atau berada pada Node terakhir, sehingga seluruh n Node harus diperiksa.

5.2 Quick Sort

5.2.1 Alasan Pemilihan

Quick Sort dipilih sebagai algoritma pengurutan untuk mengurutkan inventori berdasarkan harga karena beberapa alasan yang mendasar. Dari sisi performa, Quick Sort memiliki kompleksitas rata-rata $O(n \log n)$, yang secara signifikan lebih efisien dibandingkan Bubble Sort yang selalu beroperasi pada $O(n^2)$. Sebagai ilustrasi perbandingan, untuk 100 elemen data, Quick Sort memerlukan sekitar 664 operasi, sedangkan Bubble Sort memerlukan 10.000 operasi. Perbedaan ini semakin besar secara proporsional seiring bertambahnya jumlah data.

Dari sisi implementasi pada Linked List, Quick Sort dapat diterapkan secara in-place tanpa memerlukan alokasi memori tambahan yang signifikan, yaitu hanya stack rekursi $O(\log n)$. Hal ini lebih efisien dibandingkan Merge Sort yang memerlukan alokasi memori tambahan $O(n)$ untuk array sementara.

5.2.2 Cara Kerja

Implementasi Quick Sort pada sistem ini menggunakan strategi divide and conquer dengan Node terakhir sebagai pivot pada setiap tahap rekursi. Tahapannya adalah sebagai berikut:

Tahap	Nama	Deskripsi
1	Pemilihan Pivot	Setiap pemanggilan fungsi quickSort() menetapkan Node terakhir dari sub-list yang sedang diproses sebagai pivot.
2	Partisi	Fungsi partisi() menelusuri seluruh Node dari Node awal hingga sebelum Node pivot. Setiap Node yang nilai harganya $\leq$ harga pivot akan ditukar posisi datanya dengan Node di garis batas. Setelah penelusuran selesai, pivot ditempatkan pada posisi akhirnya.
3	Rekursi	Fungsi quickSort() dipanggil secara rekursif untuk sub-list di sebelah kiri pivot (elemen lebih kecil) dan sub-list di sebelah kanan pivot (elemen lebih besar). Basis rekursi tercapai ketika sub-list memiliki nol atau satu elemen.

5.2.3 Analisis Kompleksitas

Kasus	Kompleksitas Waktu	Kompleksitas Ruang	Kondisi
Best Case	$O(n \log n)$	$O(\log n)$	Pivot selalu membagi list menjadi dua bagian yang seimbang pada setiap tahap rekursi.
Average Case	$O(n \log n)$	$O(\log n)$	Secara statistik untuk data acak, pivot cenderung menghasilkan partisi yang cukup seimbang.
Worst Case	$O(n^2)$	$O(n)$	Data sudah terurut (ascending/descending) sehingga pivot selalu merupakan elemen terkecil atau terbesar,

Kasus	Kompleksitas Waktu	Kompleksitas Ruang	Kondisi
			menghasilkan kedalaman rekursi n.

BAB 6. PARADIGMA PEMROGRAMAN BERORIENTASI OBJEK

6.1 Abstraksi

Abstraksi adalah pilar OOP yang berkaitan dengan menyembunyikan detail implementasi dan penggambaran hanya antarmuka yang esensial. Dalam C++, abstraksi dicapai melalui abstract class yang memiliki setidaknya satu pure virtual function, yaitu fungsi virtual yang ditandai dengan = 0 dan wajib diimplementasikan oleh setiap subclass.

Dalam sistem TokoNet, kelas Item berperan sebagai abstract class. Kelas ini mendefinisikan kontrak bahwa setiap jenis barang wajib mampu menampilkan informasinya sendiri melalui metode tampilInfo() dan menghitung diskon yang berlaku melalui metode hitungDiskon(). Kedua metode ini dideklarasikan sebagai pure virtual (dengan tanda = 0), yang berarti kelas Item sendiri tidak memiliki implementasi untuk keduanya.

Konsekuensi dari desain ini adalah kelas Item tidak dapat diinstansiasi secara langsung. Objek nyata hanya dapat dibuat dari subclass konkretnya, yaitu Makanan, Elektronik, atau Minuman. Dengan demikian, abstract class Item berfungsi sebagai template atau cetakan yang memastikan bahwa setiap jenis barang dalam sistem pasti memiliki perilaku yang sudah terdefinisi sebelumnya.

Selain kedua pure virtual method tersebut, kelas Item juga mendefinisikan metode virtual getTipe() dan destruktur virtual ~Item(). Destruktor virtual merupakan keharusan dalam abstract class yang mengelola memori dinamis, karena tanpanya operasi delete pada pointer Item* hanya akan memanggil destruktur Item saja tanpa memanggil destruktur subclass, yang berpotensi menyebabkan memory leak.

6.2 Enkapsulasi

Enkapsulasi adalah pilar OOP yang berkaitan dengan pembungkusan data dan perilaku dalam satu unit serta pembatasan akses langsung terhadap data dari luar kelas. Prinsip ini diimplementasikan dalam sistem TokoNet melalui penggunaan access modifier private pada seluruh atribut kelas Item.

Atribut nama, harga, dan stok dideklarasikan sebagai private. Akses terhadap ketiga atribut ini hanya dimungkinkan melalui antarmuka publik, yaitu getter getName(), getHarga(), dan getStok() untuk operasi pembacaan, serta setter setStok() untuk operasi penulisan. Perlu dicatat bahwa hanya stok yang memiliki setter karena hanya stok yang perlu dimodifikasi setelah objek dibuat (dalam rangka transaksi penjualan), sedangkan nama dan harga bersifat tetap.

Prinsip enkapsulasi juga diterapkan pada kelas LinkedList di mana pointer head dan variabel ukuran dideklarasikan sebagai private. Hal ini mencegah kode luar memanipulasi struktur internal Linked List secara langsung yang dapat merusak konsistensi data.

6.3 Pewarisan

Pewarisan adalah pilar OOP yang memungkinkan sebuah kelas turunan (subclass) mewarisi seluruh atribut dan metode dari kelas induk (superclass) sekaligus menambahkan atribut dan metode spesifik miliknya sendiri. Hubungan ini mengekspresikan relasi "IS-A", di mana setiap Makanan adalah sebuah Item, setiap Elektronik adalah sebuah Item, dan setiap Minuman adalah sebuah Item.

Dalam sistem TokoNet, terdapat tiga subclass yang mewarisi kelas Item:

Subclass	Atribut Tambahan	Nilai hitungDiskon()	Keterangan
Makanan	kadaluarsa (string)	5.0%	Atribut menyimpan tanggal kadaluarsa produk. Implementasi tampilInfo() menyertakan informasi kadaluarsa.
Elektronik	garansiBulan (int)	10.0%	Atribut menyimpan durasi garansi dalam satuan bulan. Implementasi tampilInfo() menyertakan informasi garansi.
Minuman	dingin (bool)	3.0%	Atribut menandai apakah minuman perlu disimpan dalam kondisi dingin. Implementasi tampilInfo() menyertakan informasi tersebut.

Setiap constructor subclass wajib memanggil constructor kelas induk melalui member initializer list dengan sintaks : Item(nama, harga, stok) sebelum menginisialisasi atributnya sendiri. Kata kunci override pada setiap implementasi metode virtual di subclass berfungsi sebagai konfirmasi eksplisit kepada compiler

bahwa metode tersebut memang dimaksudkan untuk mengisi pure virtual method dari kelas induk.

6.4 Polimorfisme

Polimorfisme adalah pilar OOP yang memungkinkan objek-objek dari kelas-kelas yang berbeda diperlakukan melalui antarmuka yang sama, dengan perilaku yang berbeda-beda bergantung pada tipe objek sebenarnya pada saat program dijalankan (runtime). Dalam C++, mekanisme ini dikenal sebagai dynamic dispatch atau late binding dan dicapai melalui penggunaan virtual function dan pointer ke kelas induk.

Polimorfisme dalam sistem TokoNet terwujud secara nyata dalam cara Linked List menyimpan dan memproses data. Kelas LinkedList menyimpan seluruh barang sebagai Node yang berisi pointer Item*, yang dapat menunjuk ke objek bertipe Makanan, Elektronik, maupun Minuman. Ketika fungsi prosesJual() di server memanggil item->hitungDiskon() melalui pointer Item*, compiler C++ secara otomatis mendistribusikan panggilan tersebut ke implementasi yang tepat berdasarkan tipe objek sebenarnya:

Tipe Objek Sebenarnya	Nilai yang Dikembalikan hitungDiskon()
Makanan	5.0
Elektronik	10.0
Minuman	3.0

Manfaat konkret dari polimorfisme dalam sistem ini adalah eliminasi kebutuhan akan struktur kondisional if-else yang panjang untuk memeriksa tipe barang sebelum menghitung diskon. Kode penghitungan total harga pada fungsi prosesJual() dapat ditulis secara generik:

```
double diskon = item->hitungDiskon();  
double total = item->getHarga() * jumlah * (1.0 - diskon /  
100.0);
```

Kode ini berlaku untuk semua tipe barang tanpa modifikasi apapun. Ketika jenis barang baru ditambahkan ke sistem di masa mendatang, cukup dengan membuat subclass baru dari Item dan mengimplementasikan hitungDiskon() yang sesuai, tanpa perlu mengubah kode prosesJual() sama sekali.

BAB 7. MULTITHREADING DAN SINKRONISASI

7.1 Kebutuhan Multithreading

Sebuah server yang hanya menggunakan satu thread (single-threaded) memiliki keterbatasan mendasar: server hanya dapat melayani satu client pada satu waktu. Selama server sedang memproses permintaan client pertama, client kedua yang mencoba terhubung harus menunggu hingga seluruh interaksi dengan client

pertama selesai. Dalam konteks sistem kasir toko dengan beberapa titik layanan, keterbatasan ini menjadi bottleneck yang tidak dapat diterima.

Untuk mengatasi keterbatasan tersebut, server TokoNet mengimplementasikan model multithreading di mana setiap client yang terhubung dilayani oleh thread yang berdedikasi. Dengan pendekatan ini, kasir-kasir yang terhubung secara bersamaan dapat mengirimkan dan menerima respons secara paralel tanpa saling menunggu.

7.2 Implementasi Thread per Client

Mekanisme multithreading diimplementasikan menggunakan pustaka `std::thread` dari C++11 standard library. Setiap kali fungsi `accept()` pada `main()` menerima koneksi client baru, sebuah objek `std::thread` dibuat dengan fungsi `handleClient()` sebagai fungsi yang akan dieksekusi, beserta parameter `clientSocket` dan `clientAddr` sebagai argumen. Setelah thread dibuat, metode `detach()` segera dipanggil pada thread tersebut.

Pemanggilan `detach()` memiliki implikasi yang signifikan: thread yang di-`detach` berjalan secara mandiri tanpa perlu ditunggu oleh thread utama/main. Thread utama langsung kembali ke loop dan menjalankan `accept()` kembali untuk siap menerima koneksi berikutnya. Sumber daya yang digunakan oleh thread yang di-`detach` akan dibersihkan secara otomatis oleh sistem operasi ketika thread tersebut selesai.

Sebagai perbandingan: alternatif `join()` akan membuat thread utama menunggu hingga thread yang di-`join` selesai sebelum melanjutkan eksekusi, yang secara efektif membuat server kembali menjadi single-threaded.

7.3 Race Condition dan Kebutuhan Sinkronisasi

Multithreading memperkenalkan masalah baru yang tidak ada pada program single-threaded, yaitu race condition. Race condition terjadi ketika dua atau lebih thread mengakses dan memodifikasi data yang sama secara bersamaan tanpa koordinasi, menghasilkan hasil yang tidak terduga dan tidak konsisten.

Contoh konkret race condition dalam sistem TokoNet adalah sebagai berikut. Misalkan stok Aqua 600ml saat ini adalah 1 botol. Thread A (melayani Kasir A) dan Thread B (melayani Kasir B) keduanya secara bersamaan menerima permintaan untuk menjual 1 botol Aqua:

Urutan	Thread A (Kasir A)	Thread B (Kasir B)	Stok di Memori
1	Membaca stok → dapat nilai 1 (cukup)	-	1

Urutan	Thread A (Kasir A)	Thread B (Kasir B)	Stok di Memori
2	-	Membaca stok → dapat nilai 1 (cukup)	1
3	Mengurangi stok: $1 - 1 = 0$, menyimpan	-	0
4	-	Mengurangi stok: $1 - 1 = 0$, menyimpan	0 (SALAH, seharusnya gagal)
5 (Hasil)	Transaksi sukses	Transaksi sukses (BUG!)	0 (1 botol dijual 2x)

7.4 Mutex sebagai Mekanisme Sinkronisasi

Untuk mencegah race condition, sistem TokoNet menggunakan `std::mutex` (mutual exclusion) sebagai mekanisme sinkronisasi. Mutex adalah primitif sinkronisasi yang memastikan hanya satu thread yang dapat berada di dalam critical section (bagian kode yang mengakses data bersama) pada satu waktu.

Variabel mutex `mtx` dideklarasikan sebagai variabel global agar dapat diakses oleh semua thread. Setiap fungsi yang mengakses atau memodifikasi data inventori (`prosesLihat`, `prosesLihatUrut`, dan `prosesJual`) menggunakan `std::lock_guard<std::mutex>` untuk mengunci mutex pada awal eksekusi.

Implementasi menggunakan `lock_guard` dipilih dibandingkan pemanggilan `mutex.lock()` dan `mutex.unlock()` secara manual karena `lock_guard` mengikuti prinsip RAII (Resource Acquisition Is Initialization). Kunci mutex otomatis dilepaskan ketika objek `lock_guard` keluar dari scope, baik melalui return normal maupun melalui exception. Pendekatan manual berisiko menyebabkan deadlock apabila programmer lupa memanggil `unlock()` atau apabila exception terjadi sebelum `unlock()` dipanggil.

Perlu dicatat bahwa mutex dikunci bahkan pada operasi baca seperti `prosesLihat()`. Hal ini diperlukan untuk mencegah dirty read, yaitu kondisi di mana satu thread membaca data yang sedang dalam proses dimodifikasi oleh thread lain, yang dapat menghasilkan bacaan data yang tidak konsisten.

BAB 8 DIAGRAM ALUR SISTEM

Bab ini menyajikan diagram alur (flowchart) yang menggambarkan cara kerja sistem TokoNet secara visual, mencakup alur kerja server, alur kerja client, dan alur detail transaksi penjualan.

8.1 Alur Kerja Server

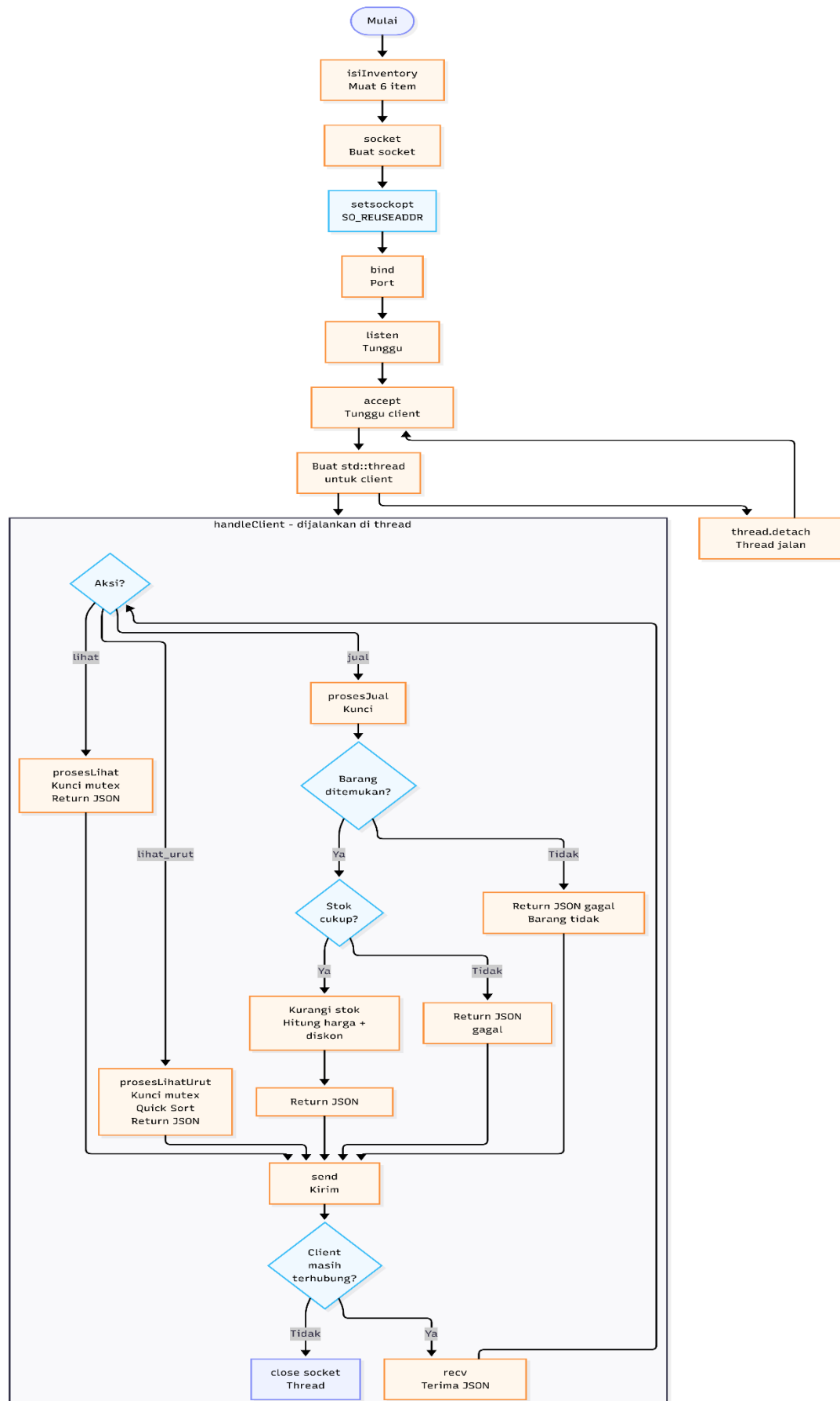


Diagram di atas menggambarkan keseluruhan siklus hidup program server dari awal dijalankan hingga melayani client. Server diawali dengan memuat data inventori awal melalui fungsi `isiInventory()`, kemudian menginisialisasi socket TCP melalui urutan wajib `socket()`, `setsockopt()`, `bind()`, dan `listen()`. Setelah siap, server memasuki loop tak terbatas pada fungsi `accept()` untuk menunggu koneksi masuk. Setiap client baru yang terhubung akan langsung dibuatkan thread tersendiri yang menjalankan fungsi `handleClient()`, sementara loop utama kembali ke `accept()` untuk melayani client berikutnya. Di dalam `handleClient()`, server menerima JSON request melalui `recv()`, menentukan jenis aksi yang diminta, meneruskan ke fungsi pemroses yang sesuai, dan mengirimkan JSON response melalui `send()`. Loop ini berlanjut hingga client memutuskan koneksi.

8.2 Alur Kerja Client

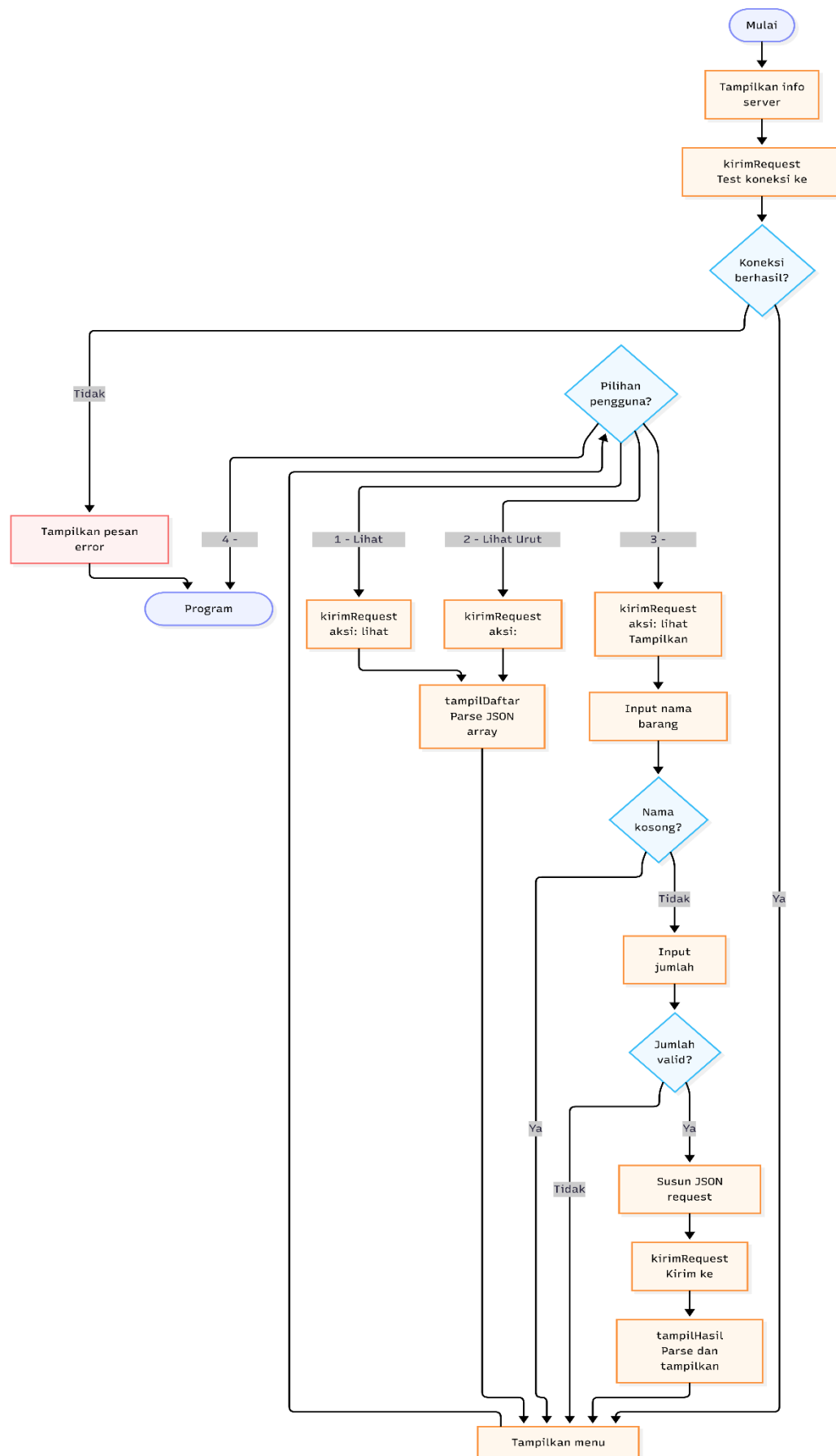


Diagram di atas menggambarkan alur eksekusi program client dari inisialisasi hingga penghentian. Setelah program dijalankan, client langsung melakukan pengecekan koneksi ke server dengan mengirimkan request lihat sebagai uji coba. Apabila koneksi gagal, program menampilkan pesan error dan berhenti. Apabila berhasil, program menampilkan menu utama dan memasuki loop interaktif. Untuk menu lihat barang dan lihaturut harga, client mengirimkan request yang sesuai dan menampilkan respons dalam format tabel. Untuk menu transaksi penjualan, client terlebih dahulu menampilkan daftar barang sebagai referensi, kemudian meminta input nama barang menggunakan `getline()` agar dapat menangani nama yang mengandung spasi, dan meminta input jumlah. Setelah input valid, client menyusun JSON request dan menampilkan hasil transaksi yang diterima dari server.

8.3 Alur Transaksi Penjualan

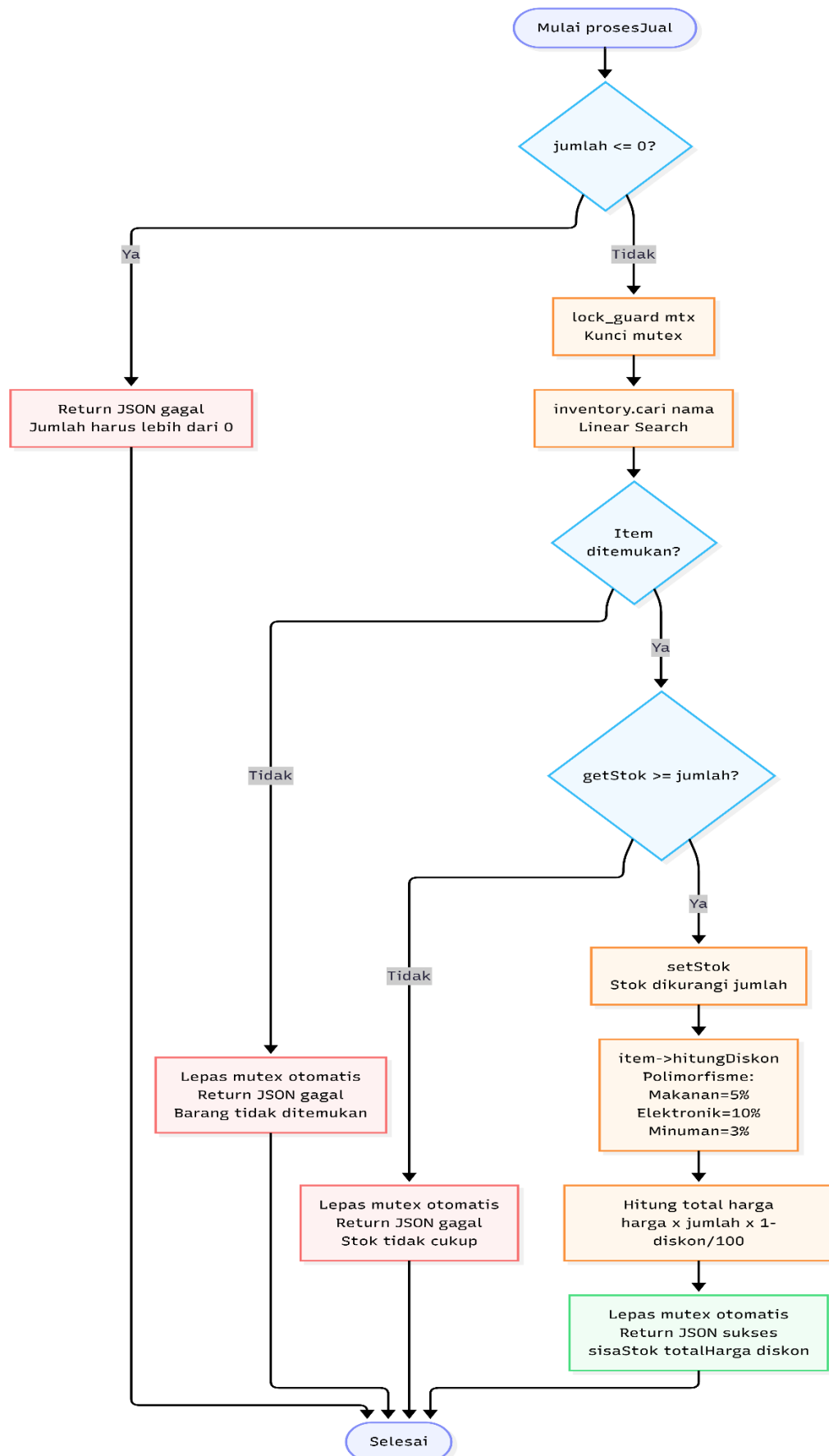


Diagram di atas menggambarkan alur detail fungsi prosesJual() yang dieksekusi di sisi server setiap kali menerima request dengan aksi "jual". Fungsi ini pertama-tama memvalidasi bahwa jumlah yang diminta lebih dari nol. Kemudian mutex dikunci menggunakan lock_guard untuk memastikan thread-safety selama proses berlangsung. Setelah mutex terkunci, fungsi memanggil inventory.cari() yang mengimplementasikan Linear Search untuk menemukan barang berdasarkan nama. Apabila barang tidak ditemukan atau stok tidak mencukupi, fungsi segera mengembalikan JSON response gagal dan mutex otomatis dilepas. Apabila semua validasi lolos, stok barang dikurangi, diskon dihitung melalui pemanggilan hitungDiskon() yang memanfaatkan polimorfisme, total harga dikalkulasi, dan JSON response sukses dikembalikan. Mutex dilepas secara otomatis saat fungsi selesai karena menggunakan lock_guard.

BAB 9 KESIMPULAN

Proyek TokoNet telah berhasil diimplementasikan sebagai sistem manajemen toko berbasis client-server yang memenuhi seluruh persyaratan teknis wajib maupun ketentuan bonus yang ditetapkan dalam rubrik penilaian.

Dari aspek struktur data, Linked List diimplementasikan secara manual tanpa menggunakan pustaka standar, mencakup operasi tambah, cari, urutkan, dan manajemen memori melalui destruktur yang eksplisit.

Dari aspek algoritma, Linear Search diimplementasikan sebagai metode pencarian yang tepat untuk Linked List mengingat keterbatasan random access struktur data tersebut, sementara Quick Sort dipilih sebagai algoritma pengurutan karena kompleksitas rata-ratanya $O(n \log n)$ yang jauh lebih efisien dibandingkan alternatif seperti Bubble Sort. Seluruh algoritma disertai analisis kompleksitas lengkap dalam notasi Big O untuk kasus terbaik, rata-rata, dan terburuk.

Dari aspek paradigma OOP, keempat pilar diimplementasikan secara terintegrasi dan saling mendukung. Abstraksi melalui abstract class Item mendefinisikan kontrak yang harus dipenuhi semua jenis barang. Enkapsulasi melindungi integritas data melalui access modifier private dan antarmuka getter/setter. Pewarisan membangun hierarki kelas yang logis antara Item dan subclass-subclassnya. Polimorfisme memungkinkan penulisan kode yang generik dan extensible melalui dynamic dispatch pada virtual functions.

Dari aspek jaringan, komunikasi client-server menggunakan TCP Socket diimplementasikan dengan urutan prosedur yang benar, dengan format pertukaran data JSON yang disusun secara manual menggunakan operasi string standar tanpa pustaka eksternal.

Dari aspek bonus, ketiga kriteria bonus terpenuhi: Linked List diimplementasikan secara manual, analisis Big O disertakan untuk setiap algoritma, dan server mengimplementasikan multithreading dengan sinkronisasi mutex yang mencegah race condition pada akses data bersama secara bersamaan.

Secara keseluruhan, proyek ini berhasil mengintegrasikan konsep-konsep fundamental mata kuliah Algoritma dan Pemrograman dalam sebuah sistem yang fungsional, koheren, dan dapat didemonstrasikan secara langsung.

LINK VIDEO:

[https://drive.google.com/drive/folders/1zC6e8szAl6vywyHiJ4EQwGGwwUGGIarx?usp=drive link](https://drive.google.com/drive/folders/1zC6e8szAl6vywyHiJ4EQwGGwwUGGIarx?usp=drive_link)

LINK GITHUB:

<https://github.com/altaficoftn/TokoNet>